Modern Computing and Python Programming
(26ESCC106)

Manjunath Prasad H. R.
Assistant Professor
Department of Artificial Intelligence and Machine Learning

Mangalore Institute of Technology and Engineering



Outline

1. Course Description
2. Course Module Topics
   - Module Topics
   - Module Lab Component
3. Course Outcomes
4. Textbooks
   - Reference Books and Web Links
5. Module 2: Python Fundamentals
   - About Python
   - Program Structure
   - Literals, Variables, Identifiers, and Keywords
   - Input and Output Statements
   - Data Types
   - Operators and Expressions
   - Type Conversion
   - Basic Debugging Techniques
   - Laboratory Programs
   - Summary and Review

Course at a Glance

Semester	**I / II**	CIE Marks	**50**
Course Code	**26ESCC106**	SEE Marks	**50**
Teaching Hrs./Week (L:T:P:S)	**3:0:2:0**	Exam Hrs.	**03**
Total Hrs	**64 (40L + 24P)**	Credits	**04**

Course Description

1. An **introductory course** on modern computing approaches, Python programming, and Artificial Intelligence & Machine Learning.
2. Begins with the **evolution of computing** and contemporary paradigms — parallel, distributed, cluster, grid, cloud, and edge computing.
3. Covers essential **Python programming constructs**, problem-solving techniques, and file processing.
4. Develops basic applications using appropriate **Python libraries** (NumPy, Pandas, Scikit-learn).
5. Concludes with an overview of **Artificial Intelligence**, intelligent agents, and the basic **Machine Learning workflow**.

Course Objectives

This course is designed to:

1. Impart knowledge on **modern computing approaches** including parallel, distributed, cluster, grid, cloud, and edge computing.
2. Impart knowledge on Python **data types, operators**, and **input/output** statements.
3. Provide knowledge of **strings, control structures, built-in data structures, functions**, and **file handling** in Python.
4. Provide fundamentals of **artificial intelligence** and **machine learning** techniques.

Course Module 1 Topics - Modern Computing Approaches (8L)

- **Evolution of Computing:** Scalable computing over the Internet, the Age of Internet Computing, Scalable Computing Trends and New Paradigms
- **Parallel and Distributed Computing:** Eras of computing, parallel vs. distributed computing, parallel processing, elements of distributed computing
- **Cluster Computing:** Introduction, why clusters exist, components and characteristics, types, examples
- **Grid Computing:** Definition, power grid analogy, virtual organisation, characteristics, types, examples, advantages and limitations
- **Cloud Computing:** Introduction, cloud reference model (IaaS, PaaS, SaaS), types of clouds (public, private, hybrid, community)
- **Edge Computing:** Introduction, need, architecture, cloud vs. edge, applications

Course Module 2 Topics - Python Fundamentals (8L + 6P)

- About Python, Program Structure
- Variables, Keywords, Identifiers, Literals
- Input and Output Statements
- Data Types, Operators, Expressions
- Operator Precedence, Type Conversion
- Basic Debugging Techniques

Module 2 Laboratory - Python Basics

- Read student name, USN, and marks of three courses; compute total and average; display in a formatted way.
- Read the radius of a circle; compute area and circumference; convert the area to an integer.
- Accept an integer, a float, a string, and a Boolean; display values and data types; perform arithmetic and string operations.
- Accept two numbers and perform addition, multiplication, division, floor division, modulus, and exponentiation.

Course Module 3 Topics - Strings, Flow Control, and Functions (8L + 6P)

- **Strings:** Creating and storing strings, built-in string functions, string operators, indexing & slicing, formatting (f-strings & `format()`)
- **Flow Control:** `if`, `if-else`, `elif`; `for`, `while`; nested loops; `break`, `continue`, `pass`
- **Functions:** Defining & calling, built-in and user-defined functions, arguments & parameters, global and local variables, command line arguments, lambda functions, importing user-defined modules

Module 3 Laboratory - Strings, Menus, and Functions

- String operations: indexing, slicing, case conversion, counting, replacing, split/join, `startswith`/`endswith`.
- Menu-driven program: even/odd, largest of three, multiplication table, primes in a range, exit.
- Factorial using a function; simple interest with default arguments; square using a lambda.
- Create a module `mymath.py` with arithmetic functions and import it into another program.

Course Module 4 Topics - Built-in Data Structures and Files (8L + 6P)

- **Python Data Structures:** Mutable and immutable objects, lists, tuples, sets, dictionaries
- Common operations on lists, tuples, and dictionaries
- Passing lists, tuples, and dictionaries to functions
- **File Handling:** Introduction to files, opening a file, file modes, reading, writing, appending, closing a file

Module 4 Laboratory - Data Structures and Files

- List operations: insert, delete, sort, reverse; tuple of names: display, count, index.
- Dictionary of USN $\rightarrow$ name: add, update, delete, display; functions accepting list, tuple, and dictionary.
- Text files: write, append, read; count characters, words, and lines; `read()`, `readline()`, `readlines()`.
- File copy: copy, verify, display contents and sizes of both files.
- Tuple of integers (max, min, count, index); set operations; Employee ID dictionary operations.

Course Module 5 Topics - Artificial Intelligence and Machine Learning (8L + 6P)

- **Artificial Intelligence:** Introduction, history and evolution of AI, applications of AI
- **Intelligent Agents:** Agents and environments, structure of agents
- **Machine Learning:** Introduction; supervised, unsupervised, and reinforcement learning
- **ML Workflow / Pipeline:** Structure of data, data pre-processing, features, feature selection, model selection, training, testing, and evaluation

Module 5 Laboratory - Working with a Dataset

- Load the Titanic dataset (`Titanic.csv`) using Pandas.
- Display the first five records, dataset information, and summary statistics.
- Select and display the input features and the target attribute.
- Split the dataset into training and testing sets using Scikit-learn.
- Display the number of training and testing samples.

Course Outcomes

At the end of this course, the student will be able to:

1. **DESCRIBE** modern computing approaches and their characteristics.
2. **DESCRIBE** fundamental Python programming constructs.
3. **DESCRIBE** the basics of artificial intelligence and machine learning.
4. **WRITE** Python programs to solve simple computational problems and perform various file operations.
5. **IDENTIFY** suitable Python built-in data structures and utilize them for problem solving.
6. **DEVELOP** Python programs utilizing NumPy, Pandas, and Scikit-learn to process given datasets.

Textbooks

1. Kai Hwang, Geoffrey C. Fox and Jack J. Dongarra, *Distributed and Cloud Computing: From Parallel Processing to the Internet of Things*, Morgan Kaufmann, Elsevier, 2012.
2. Rajkumar Buyya, Christian Vecchiola, S. Thamarai Selvi, *Mastering Cloud Computing*, McGraw Hill Education, 2013.
3. Reema Thareja, *Python Programming using Problem Solving Approach*, Oxford University Press, 3rd Edition, 2024.
4. Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, 4th Edition, Pearson, 2021.
5. Saikat Dutt, Subramanian Chandramouli, Amit Kumar Das, *Machine Learning*, Pearson Education India, 1st Edition, 2018.

Reference Books and Web Links

Reference Books:

1. Perry Lea, *IoT and Edge Computing for Architects*, Packt Publishing, 2nd Edition, 2020.
2. Eric Matthes, *Python Crash Course*, No Starch Press, 2015.
3. Al Sweigart, *Automate the Boring Stuff with Python*, No Starch Press, 2019.
4. Mark Fenner, *Machine Learning with Python for Everyone*, Addison-Wesley Professional, 2020.

Web Links (NPTEL):

- Python full course: `https://nptel.ac.in/courses/106106182`
- Fundamentals of AI: `https://nptel.ac.in/courses/112103280`

Tools Used in this Course

Category	**Tools**
Programming Language	Python 3
Numerical Computing	NumPy
Data Handling	Pandas
Machine Learning	Scikit-learn
Dataset (Lab)	Titanic dataset (`Titanic.csv`)

Thank You

Module 2 - Python Fundamentals: What We Will Learn

1. **About Python** — features, history, applications, and limitations.
2. **Program Structure** — writing and running a program, comments, indentation.
3. **Building Blocks** — literals, variables, identifiers, and keywords.
4. **Input and Output** — `input()` and `print()`.
5. **Data Types** — numbers, strings, and Boolean values.
6. **Operators and Expressions** — operator precedence and associativity.
7. **Type Conversion** and **Basic Debugging Techniques**.
8. **Laboratory Programs** — four programs for this module.

What Is Python?

Python

A **high-level**, **interpreted**, **interactive**, **object-oriented**, and reliable programming language that uses simple **English-like words**.

1. It has a very simple syntax, so beginners can start writing programs quickly.
2. It has a **vast library of modules** that lets us build complex solutions from ready-made components.
3. It is an **open-source** project, supported by a large community of users.
4. It is **platform-independent**: the same program runs on Windows, Linux, and macOS.
5. **Used by:** Google, YouTube, NASA, Instagram, Netflix, and most AI/ML research.

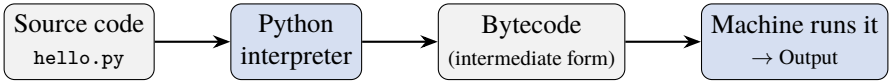
Features of Python (1/2)

Feature	**Meaning**
Simple	Reading a Python program feels almost like reading English; we focus on the problem, not the language.
Easy to learn	Few keywords and a clearly defined syntax.
Versatile	Used for everything from simple text processing to web browsers and games.
Free and open source	Anyone can freely use, read, change, and share it.
High-level language	No need to worry about low-level details such as managing memory.
Interactive	We can type a command at the >>> prompt and see the result immediately.
Portable	Programs run on many platforms (Linux, Windows, macOS, …) without changes.
Object oriented	Supports both object-oriented and procedure-oriented programming.

Features of Python (2/2)

Feature	**Meaning**
Interpreted	No separate compile step; the interpreter runs the program directly (through **bytecode**).
Dynamic	Programs run dynamically; errors are reported at run time, which helps interactive development.
Extensible	Low-level modules (in C or C++) can be added to the Python interpreter.
Embeddable	Python can be embedded in C, C++, Java, and other programs to add scripting.
Extensive libraries	A huge library for text processing, databases, GUI programming, and more.
Easy maintenance; secure	Code is easy to maintain; modules can be distributed to prevent tampering.
Robust	Errors are raised as **exceptions**, which the program can catch and handle.
Multi-threaded; garbage collection	Runs several tasks together; unused memory is freed automatically.

How Python Runs Your Program



1. Python is processed at **run time** by the **interpreter**; there is no separate compile step.
2. The interpreter converts the source code into an intermediate form called **bytecode**, which is then translated into the native language of the computer.
3. Bytecode makes Python **portable**: copy the program to another computer and run it — no compiling, linking, or loading.
4. **Analogy:** An interpreter is like a live translator at a meeting who translates each sentence as it is spoken.

Limitations of Python

1. **Slower than C/C++:** Being interpreted, Python is slower, especially for heavy computation.
2. **Not ideal for high-graphic 3D games** that use a lot of CPU.
3. **Parallel processing** is not as elegant as in some other languages (such as Go).
4. **Fewer users than C, C++, or Java** (at the time the textbook was written) and limited commercial support.
5. **Hard to package** a big Python application into a single executable file for non-technical users.
6. **Still evolving**, so some documentation may lag behind.

History of Python

1. Developed by **Guido van Rossum** in the late 1980s and early 1990s at the **National Research Institute for Mathematics and Computer Science (CWI)** in the Netherlands.
2. Derived from many languages: ABC, Modula-3, C, C++, Algol-68, SmallTalk, and Unix shell.
3. **Released in February 1991.** Python 2.0 came in 2000; **Python 3** (used in this course) was released in 2008.
4. Its source code is available under an open license and is maintained by a core development team.
5. **Why the name "Python"?** Guido was reading scripts of **Monty Python's Flying Circus**, a BBC comedy series. He wanted a name that was short, unique, and slightly mysterious.
6. **Fun fact:** Python is older than Java, R, and JavaScript.

Applications of Python

Area	Examples
Web development	Frameworks Django, Flask; websites such as Quora and Reddit
Scientific and computational	NumPy, SciPy, Pandas; FreeCAD; finite element software
GUI desktop applications	Toolkits wxPython, PyQt, PyGtk
Image processing and graphics	GIMP, Inkscape, Blender, Maya (for automation)
Games	PyGame library; parts of Civilization-IV
Operating systems	Ubuntu's Ubiquity installer; Red Hat's Anaconda installer
Embedded scripting, prototyping, networking	Testing and build tools; quick prototypes; FTP, IMAP, e-mail programs
Teaching	A perfect first programming language

1. **Today:** Python is the leading language for **Artificial Intelligence and Machine Learning** (Module 5 of this course).

The Future of Python

1. Python has a **huge, constantly growing** user base; it is a stable language that is here to stay.
2. It is a preferred language of companies such as Google, YouTube, and Nokia, and of **NASA**, for its easy syntax.
3. It supports multiple programming styles: **object-oriented**, **functional**, and **parallel** programming.
4. It is among the **top five** most preferred languages in academia and industry.
5. It is used for everything from photo applications to social networks, automation, and science.

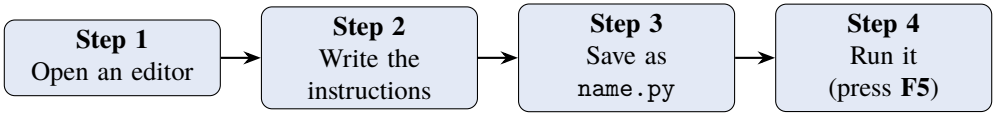
Writing and Executing the First Python Program

1. Download Python from `www.python.org`. It comes with **IDLE** (Integrated Development and Learning Environment).
2. IDLE shows the **Python prompt** `>>>`. Type an instruction and press **Enter**:

```
>>> print("Hello World!!!")
Hello World!!!
```

3. **Interactive mode (shell):** one statement at a time; the result is shown immediately. Good for trying things out.
4. **Script mode (file):** write the whole program in a file, save it, and run it. Good for real programs.
5. To exit IDLE: **File** → **Exit**, or press **Ctrl + Q**, or type `quit()`.

Writing Python Programs: Four Steps



1. In IDLE: **File** → **New File**, type the program, save it with the extension `.py`.
2. Run it with **F5** or **Run** → **Run Module**; the output appears in the shell window.
3. From the command prompt: `python program_name.py`
4. **Common mistake:** forgetting to save, or saving without `.py` — then IDLE cannot run the file as a Python program.

Structure of a Python Program

```
# Program to find the area of a rectangle
# Author: Student of MITE

# Input
length = float(input("Enter length: "))
width  = float(input("Enter width: "))

# Processing
area = length * width

# Output
print("Area =", area)
```

1. **Comments** (#) describe the program; Python ignores them.
2. **Input:** read data from the user.
3. **Processing:** calculate using variables, operators, and expressions.
4. **Output:** display the result.
5. Statements run **from top to bottom**, one after another.

Almost every program we write in this module follows: Input → Processing → Output.

Comments

Comment

A **non-executable** statement added to explain the program. In Python, a # sign (not inside a string) begins a comment; everything after it up to the end of the line is ignored.

```
# This is a comment
print("Hello")   # to display hello
# Program ends here
```

Output

```
Hello
```

1. Comments make the program easier to read and understand — for others and for you later.
2. A comment can be on its own line or at the end of a statement.
3. A program can have any number of comments.

Indentation

1. **Indentation** is the whitespace (spaces or tabs) at the **beginning** of a line.
2. In most languages it only makes code look neat. In Python, indentation is used to **group statements into blocks** (instead of curly braces { }).
3. All statements in a block must have the **same indentation**. A wrong indent is an error:

```
age = 21
    print("You can vote")
```

Output

```
    print("You can vote")
IndentationError: unexpected indent
```

4. For now, start every statement at the **left margin**. Blocks are used with `if` and loops in Module 3.

Physical Lines, Logical Lines, and Statements

1. A **physical line** is what we see in the editor; a **logical line** is what Python sees as one statement.
2. Normally, **one statement per line** — this makes code readable.
3. More than one statement on a line must be separated by a **semicolon** (`;`) — avoid it.

```
msg = "Hello"
print(msg)
```

```
msg = "Hello"; print(msg)
```

Output (both)

```
Hello
```

Literal Constants

Literal Constant

A value written **directly** in a program, such as `7`, `3.9`, `'A'`, or `"Hello"`. It is called *literal* because it means only itself, and *constant* because its value cannot be changed.

Kind	Examples	Notes
Integer	`5`, `-12`, `535633629843`	Whole numbers of any size
Floating point	`3.23`, `91.5E-2`	`E` means power of 10: `91.5E-2` $= 91.5 \times 10^{-2}$
Complex	`-3 + 7j`	In Python, the imaginary part is written with `j`
String	`'Hi'`, `"Hi"`, `'''Hi'''`	Text inside quotes
Boolean	`True`, `False`	Only two values

1. **Commas are never used** in numbers: `3,567` is not 3567 in Python.

Simple Operations on Numbers

```
>>> 10 + 7
17
>>> 96 / 12
8.0
>>> 78 // 5       # quotient
15
>>> 78 % 5        # remainder
3
>>> 5 ** 3        # power
125
```

```
>>> 5 * 3.0       # int and float
15.0
>>> 15 / 0
ZeroDivisionError: division by zero
>>> 0.1 + 0.2
0.30000000000000004
>>> format(16/3, '.2f')
'5.33'
```

1. `/` always gives a **float** (`96/12` is `8.0`).
2. Floats have **limited precision**: `0.1 + 0.2` is not exactly `0.3` (loss of precision). Very large results give `inf` (**overflow**); very small ones give `0.0` (**underflow**).
3. `format(value, '.2f')` rounds the display to 2 decimal places.

String Literals

1. A **string** is a group of characters. It can be written in:

```
>>> 'HELLO'            # single quotes
'HELLO'
>>> "HELLO"            # double quotes
'HELLO'
>>> '''Good morning
... everyone.'''       # triple quotes
'Good morning\neveryone.'
```

```
>>> print('Beautiful Weather' '.....'
...       'Seems it would rain')
Beautiful Weather.....Seems it would rain
```

2. Single and double quotes are **exactly the same**. Triple quotes allow **multi-line** strings.
3. Two string literals placed side by side are joined automatically.
4. Strings are **immutable**: once created, they cannot be changed.
5. Python has **no separate character type**: `'A'` is a string of length 1.

Escape Sequences and Raw Strings

Escape	Meaning
`\n`	New line
`\t`	Tab
`\\`	Backslash
`\'`	Single quote
`\"`	Double quote
`\a`	Bell (beep)

```
>>> print('What's your name?')
SyntaxError: unterminated string literal
>>> print('What\'s your name?')
What's your name?
>>> print("Hello\tWorld")
Hello   World
>>> print(r"What\'s your name?")
What\'s your name?
```

1. An **escape sequence** is a backslash followed by a character; it represents a character that is hard to type directly.
2. A **raw string** (prefix `r` or `R`) ignores escape sequences and prints the text exactly as written.

Variables

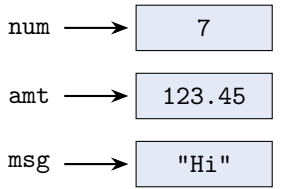
Variable

A **named, reserved memory location** that stores a value. Its value can change (vary) while the program runs.

```
num = 7
amt = 123.45
msg = "Hi"
print(num, amt, msg)
```

Output

```
7 123.45 Hi
```



memory locations

1. **Analogy:** A variable is a **labelled box**; the name is the label, the value is what is inside.
2. No declaration is needed: the variable is created when a value is first **assigned** with =.

Identifiers

Identifier

A **name** given to identify a variable, function, class, module, or other object. Variables are examples of identifiers.

Rules for naming identifiers:

1. The first character must be a **letter** (upper or lower case) or an **underscore** (`_`).
2. The rest may contain letters, underscores, and **digits** (0–9).
3. Identifiers are **case-sensitive**: `myvar` and `myVar` are different.
4. Punctuation such as `@`, `$`, `%`, spaces, and hyphens are **not allowed**.
5. A **keyword** cannot be used as an identifier.

Valid	`sum`, `__my_var`, `num1`, `r`, `var_20`, `First`, `total_marks`
Invalid	`1num` (starts with digit), `my-var` (hyphen), `%check` (symbol), `Basic Sal` (space), `H#R&A`, `class` (keyword)

Keywords (Reserved Words)

1. **Keywords** are words with a **pre-defined meaning** in Python. They cannot be used as identifiers.
2. Python 3.11 has **35 keywords**:

`False`	`None`	`True`	`and`	`as`	`assert`	`async`
`await`	`break`	`class`	`continue`	`def`	`del`	`elif`
`else`	`except`	`finally`	`for`	`from`	`global`	`if`
`import`	`in`	`is`	`lambda`	`nonlocal`	`not`	`or`
`pass`	`raise`	`return`	`try`	`while`	`with`	`yield`

3. All keywords are lowercase **except** `True`, `False`, and `None`.
4. **Note:** The textbook's table (Table 3.2) is from Python 2, where `print` and `exec` were keywords. In Python 3, `print` is a function.
5. To see the list yourself: `import keyword` and then `print(keyword.kwlist)`.

Output with `print()`

```
print("Hello, MITE!")
print("Total =", 250)
print(1, 2, 3, sep="-")
print("No new line", end="!\n")
print("Line 1\nLine 2")
```

Output

```
Hello, MITE!
Total = 250
1-2-3
No new line!
Line 1
Line 2
```

1. `print()` displays values on the screen; the quotes of a string are **not** displayed.
2. Several values separated by commas are printed with a **space** between them.
3. `sep` changes the separator; `end` changes what is printed at the end (default: a new line).

Formatted Output with `format()`

```
>>> format(16/3, '.2f')       # 2 decimals
'5.33'
>>> format(3**50, '.5e')      # scientific
'7.17898e+23'
>>> format(123456, ',')       # commas
'123,456'
>>> format('Hello', '<10')    # left
'Hello     '
>>> format('Hello', '>10')    # right
'     Hello'
>>> format('Hello', '^10')    # centre
'  Hello   '
```

1. Syntax: `format(value, format_specifier)`
2. `.2f` — round to 2 decimal places.
3. `e` — scientific notation.
4. `<`, `>`, `^` — left, right, and centre alignment in a given width.
5. Used in Lab 1 and Lab 2 to show marks and areas neatly.

Input with `input()`

```
name = input("What's your name? ")
age = input("Enter your age : ")
print(name + ", you are " + age + " years old")
```

Output

```
What's your name? Goransh
Enter your age : 10
Goransh, you are 10 years old
```

1. `input(prompt)` displays the **prompt**, waits for the user to type, and returns what was typed.
2. **Very important:** `input()` **always returns a string** — even if the user types a number.
3. So `age + 1` gives an error, because `age` is the string `"10"`, not the number 10.
4. **Note:** In Python 2 this function was called `raw_input()`.

Reading Numbers from the User

Without conversion

```
x = input("Enter the first number:")
y = input("Enter the second number:")
print(x + y)
```

Output

```
Enter the first number:6
Enter the second number:7
67
```

With `int()` conversion

```
x = int(input("Enter the first number:"))
y = int(input("Enter the second number:"))
print(x + y)
```

Output

```
Enter the first number:5
Enter the second number:6
11
```

1. On the left, `"6" + "7"` **joins** the strings to give `"67"`.
2. Use `int(input(...))` for whole numbers and `float(input(...))` for decimal numbers.

Data Types in Python

1. A **data type** tells what kind of value a variable holds and which operations are allowed on it.
2. Based on the data type, the interpreter **reserves memory** for the value.

Type	Name	Example
Integer	`int`	25, -7
Float	`float`	3.5, 1.2e3
Complex	`complex`	3+4j
String	`str`	"MITE"
Boolean	`bool`	True, False
List / Tuple / Dict	`list`, `tuple`, `dict`	Module 4

```
>>> type(7)
<class 'int'>
>>> type(3.9)
<class 'float'>
>>> type('A')
<class 'str'>
>>> type(True)
<class 'bool'>
```

3. The built-in function `type()` tells the data type of any value.

Assigning and Reassigning Values

```
val = 'Hello'
print(val)
val = 100
print(val)
val = 12.34
print(val)
```

Output

```
Hello
100
12.34
```

1. `=` is the **assignment operator**: the name on the left gets the value on the right.
2. It is **not** "equal to" as in mathematics; `x = x + 1` is valid and adds 1 to `x`.
3. A variable can be **reassigned** as many times as we want.
4. It can even hold a value of a **different type** later — Python variables do not have fixed types (**dynamic typing**).

Multiple Assignment

```
sum = flag = a = b = 0   # same value
sum, a, b, mesg = 0, 3, 5, "RESULT"
print(sum, a, b, mesg)
```

Output

```
0 3 5 RESULT
```

```
>>> print(age)
NameError: name 'age' is not defined
>>> num = 10
>>> del num
>>> print(num)
NameError: name 'num' is not defined
```

1. One value can be assigned to several variables at once, or several values to several variables.
2. Using a variable **before assigning it**, or **after deleting it** with `del`, gives a `NameError`.

Boolean Data Type

1. A **Boolean** variable has only two values: `True` or `False` (capital T and F).
2. Boolean values are produced when we **compare** values:

```
>>> 20 == 30
False
>>> 20 != 20
False
>>> 90 <= 90
True
```

```
>>> 87 == 87.0
True
>>> 30 > 50
False
>>> 87 >= 87.0
True
```

```
>>> "Python" == "Python"
True
>>> passed = True
>>> print(passed)
True
```

3. Booleans are used for **decisions** (`if` statements in Module 3).
4. `True` behaves as `1` and `False` as `0` in arithmetic: `True + True` is `2`.

Operators and Operands

Operator and Operand

An **operator** is a symbol that performs an operation; the values it works on are **operands**. In `sum = a + b`, `a` and `b` are operands and `+` is the operator.

Types of operators in Python:

1. Arithmetic operators
2. Comparison (relational) operators
3. Assignment operators
4. Logical operators
5. Unary operators
6. Bitwise and shift operators
7. Membership operators
8. Identity operators

Arithmetic Operators

Operator	Description	Example (a=100, b=200)	Output
`+`	Addition	`a + b`	`300`
`-`	Subtraction	`a - b`	`-100`
`*`	Multiplication	`a * b`	`20000`
`/`	Division (always gives a float)	`b / a`	`2.0`
`%`	Modulus: remainder of division	`b % a`	`0`
`//`	Floor division: quotient, rounded down	`12 // 5`	`2`
`**`	Exponent: power	`a ** 2`	`10000`

1. **Floor division** rounds **towards negative infinity**: `-19 // 5` is `-4`, not `-3`.
2. If either operand is a float, the result is a float: `12.0 // 5.0` is `2.0`.
3. **Everyday use:** `%` tells if a number is even (`n % 2 == 0`); `//` converts 135 minutes into 2 hours (`135 // 60`).

Comparison (Relational) Operators

Operator	Returns True if …	Example (a=100, b=200)	Output
`==`	the two values are equal	`a == b`	`False`
`!=`	the two values are not equal	`a != b`	`True`
`>`	left value is greater than right value	`a > b`	`False`
`<`	left value is less than right value	`a < b`	`True`
`>=`	left value is greater than or equal to right value	`a >= b`	`False`
`<=`	left value is less than or equal to right value	`a <= b`	`True`

1. Comparison operators compare two values and always give a **Boolean** result.
2. **Common mistake:** `=` assigns a value; `==` compares two values.
3. Strings are compared alphabetically: `"apple" < "banana"` is `True`.

Assignment and Shortcut (In-place) Operators

Operator	Same as
`a += b`	`a = a + b`
`a -= b`	`a = a - b`
`a *= b`	`a = a * b`
`a /= b`	`a = a / b`
`a %= b`	`a = a % b`
`a //= b`	`a = a // b`
`a **= b`	`a = a ** b`

```
>>> count = 10
>>> count += 5
>>> count
15
>>> str1 = "Good "
>>> str1 += "Morning"
>>> print(str1)
Good Morning
```

1. **Shortcut operators** let us write `num = num + 10` more concisely as `num += 10`.
2. They also work on other data types, e.g., `+=` on strings.
3. Python has **no** `++` or `--` operators (unlike C); use `+= 1`.

Logical Operators

A	B	A and B	A or B	not A
False	False	False	False	True
False	True	False	True	True
True	False	False	True	False
True	True	True	True	False

```
>>> marks = 72
>>> marks >= 40 and marks < 75
True
>>> marks < 40 or marks > 90
False
>>> not (marks >= 40)
False
```

1. `and`: True only if **both** conditions are true. `or`: True if **at least one** is true. `not`: reverses the value.
2. **Short-circuit:** Python stops as soon as the answer is known — in `False and …`, the right side is not evaluated.
3. **Note:** The textbook writes `&&`, `||`, and `!` (C style). In Python, write `and`, `or`, `not`.

Unary, Bitwise, and Shift Operators

Operator	Name	10 op 12
`&`	Bitwise AND	`8`
`|`	Bitwise OR	`14`
`^`	Bitwise XOR	`6`
`~`	Bitwise NOT	`~10 = -11`
`<<`	Left shift	`13 << 1 = 26`
`>>`	Right shift	`13 >> 1 = 6`

```
  10 = 1010
  12 = 1100
----------------
1010 & 1100 = 1000  (8)
1010 | 1100 = 1110  (14)
1010 ^ 1100 = 0110  (6)
```

1. **Unary minus** (`-`) acts on one operand and changes its sign: if `b = 10`, then `-b` is `-10`.
2. **Bitwise** operators work on the **binary bits** of integers; they cannot be used on floats.
3. Shifting left by 1 **doubles** a number; shifting right by 1 **halves** it (whole-number part).

Membership and Identity Operators

Membership: `in`, `not in`

```
>>> 'e' in 'hello'
True
>>> 'z' not in 'hello'
True
>>> 3 in [1, 2, 3]
True
```

Identity: `is`, `is not`

```
>>> p = [1, 2]
>>> q = p
>>> r = [1, 2]
>>> p is q, p is r, p == r
(True, False, True)
```

1. `in` returns `True` if a value is **found** in a sequence (string, list, tuple); `not in` returns `True` if it is **not** found.
2. `is` returns `True` if both operands refer to the **same object** in memory (same `id()`).
3. `==` checks if the **values** are equal; `is` checks if they are the **same object**. Here `p` and `r` have equal values but are different objects.

Expressions

Expression

Any legal combination of **operands** (variables, constants) and **operators** that produces a value.
Example: `A * B + C - 5`.

Type of Expression	Description	Example
Constant	Only constants	`8 + 9 - 2`
Integral	Gives an integer result	`c = a * b` (a, b integers)
Floating point	Gives a float result	`a * b / 2`
Relational	Gives `True` or `False`	`c = a > b`
Logical	Combines relational expressions	`a > b and y != 0`
Bitwise	Works at the bit level	`x = y & z`
Assignment	Assigns a value to a variable	`c = a + b`

1. By position of the operator: **infix** (`a + b`, used in Python), **prefix** (`+ a b`), **postfix** (`a b +`).
2. Illegal expressions such as `a+` or `-b,` produce a `SyntaxError`.

Operator Precedence

Level	Operator(s)	Name
1 (highest)	`( )`	Parentheses
2	`**`	Exponent
3	`+x`, `-x`, `~x`	Unary
4	`*`, `/`, `//`, `%`	Multiplicative
5	`+`, `-`	Additive
6	`<<`, `>>`	Shift
7	`&`, then `^`, then `|`	Bitwise
8	`== != < > <= >= is in`	Comparison, identity, membership
9	`not`	Logical NOT
10	`and`	Logical AND
11 (lowest)	`or`	Logical OR

1. When an expression has more than one operator, **precedence** decides which is evaluated first.
2. Python follows **BEDMAS**: Brackets, Exponents, Division/Multiplication, Addition/Subtraction.
3. Assignment (=, +=, …) happens **last**, after the whole right side is evaluated.
4. **When in doubt, use parentheses.**

Precedence and Associativity: Examples

```
>>> 10 + 30 * 5
160
>>> (40 + 20) * 30 / 10
180.0
>>> 40 + (20 * 30) / 10
100.0
>>> False == False or True
True
>>> False == (False or True)
False
```

```
>>> 100 / 10 / 5       # left to right
2.0
>>> 2 * 3 % 4          # left to right
2
>>> 2 ** 3 ** 2        # right to left!
512
>>> (2 ** 3) ** 2
64
```

1. **Associativity** decides the order when operators have the **same precedence**.
2. Most operators are evaluated **left to right**; `**` is evaluated **right to left**: `2**3**2` $= 2^9 = 512$.
3. `*` is done before `+`, so `10 + 30 * 5` $= 10 + 150 = 160$.

Why Do We Need Type Conversion?

```
>>> "20" + "30"
'2030'
>>> int("2") + int("3")
5
>>> "Python" + 3.4
TypeError: can only concatenate
str (not "float") to str
>>> "Python" + str(3.4)
'Python3.4'
```

1. Some operations are not possible between different types, e.g., `"2" + 4`.
2. `input()` returns a string, so numbers typed by the user must be converted before calculation.
3. **Type conversion** changes a value from one data type to another using built-in functions. These functions return a **new** value; the original is unchanged.

Type Conversion Functions

Function	Converts x to …
`int(x)`	an integer
`float(x)`	a floating-point number
`str(x)`	a string
`bool(x)`	a Boolean
`list(x)`, `tuple(x)`, `set(x)`	a list, tuple, set
`ord(x)`	character → its integer code
`chr(x)`	integer code → character
`hex(x)`, `oct(x)`	hexadecimal, octal string

```
>>> int(2.90)        # truncates
2
>>> round(2.90)      # rounds
3
>>> round(89.567890, 2)
89.57
>>> ord('A'), chr(66)
(65, 'B')
>>> hex(100), oct(100)
('0x64', '0o144')
```

1. `int()` **cuts off** the decimal part (truncation); `round()` rounds to the nearest integer.
2. `int("abc")` gives a `ValueError`: the string must look like a number. (The textbook's `long()` and `unichr()` are Python 2 only.)

Implicit vs. Explicit Conversion

Implicit conversion (type coercion)

```
>>> 21 + 2.1      # int becomes float
23.1
>>> 5 * 3.0
15.0
```

- Done **automatically** by Python.
- The smaller type is converted to the larger one, so no information is lost.

Explicit conversion (type casting)

```
>>> num = int("15")
>>> num + 3
18
>>> float(num)
15.0
```

- Done by the **programmer** using functions such as `int()`, `float()`, `str()`.
- Needed when Python cannot convert automatically.

Debugging

The process of **finding and fixing errors (bugs)** in a program. During debugging, a program is run several times with different inputs to locate errors.

Type of Error	When / What	Example
Syntax error	Breaks the rules of Python; the program does not start	`print("Hello"` — missing `)`
Run-time error (exception)	Program starts but stops in the middle	`15 / 0` → `ZeroDivisionError`
Logical error	Program runs but gives a **wrong answer**	Average written as `m1 + m2 + m3 / 3`

1. Logical errors are the hardest to find, because Python shows no error message.
2. **Testing** shows that bugs exist; it cannot prove that a program is bug-free.

Reading an Error Message (Traceback)

```
Traceback (most recent call last):
  File "marks.py", line 3, in <module>
    total = m1 + 10
TypeError: can only concatenate str (not "int") to str
```

1. A **traceback** is Python's error report. Read it from the **bottom up**.
2. **Last line:** the **type** of error and a short message — here, `TypeError`.
3. **Line above:** the file, the **line number**, and the exact statement where it happened.
4. **Fix here:** `m1` came from `input()` and is a string; convert it with `int()` or `float()`.
5. **An error message is information, not failure.** Read it before changing anything.

Common Errors for Beginners

Error	Typical Cause	Fix
`SyntaxError`	Missing bracket or quote; `Print` instead of `print`	Check brackets, quotes, spelling
`IndentationError`	Extra space or tab at the start of a line	Start statements at the same margin
`NameError`	Variable used before assigning, or misspelt	Assign first; check spelling and case
`TypeError`	Adding a string to a number	Convert with `int()`, `float()`, `str()`
`ValueError`	`int("abc")` — text that is not a number	Enter valid input
`ZeroDivisionError`	Dividing by zero	Check the divisor before dividing

1. **Frequent errors (textbook):** misspelt variable names, wrong case (creates a new variable), and forgetting to initialize a variable.
2. Curly "smart quotes" copied from WhatsApp or Word also cause `SyntaxError` — type quotes in the editor.

Debugging Aids

1. **Print statements:** Insert `print()` to see the value of variables at different points.
2. **Tracebacks:** When a run-time error occurs, Python reports the line number and cause.
3. **Assert statements:** `assert condition` stops the program with an error if the condition is `False`.
4. **Debugger:** Step through the program line by line and watch variables change.

```
m1, m2, m3 = 85, 78, 92
avg = m1 + m2 + m3 / 3
print("DEBUG avg =", avg)    # 193.66...
```

```
avg = (m1 + m2 + m3) / 3    # fixed
assert avg <= 100, "average too big"
print("Average =", avg)     # 85.0
```

5. Once an error is detected, fix its **cause**, not just the symptom.

Debugging Using the Python IDLE Debugger

1. **Step 1:** In the shell window, choose **Debug** → **Debugger**. The **Debug Control** window opens and `[DEBUG ON]` appears.
2. **Step 2:** In the program window, right-click a line and choose **Set Breakpoint**. The line turns yellow.
3. **Step 3:** Press **F5** to run. The highlighted line shows which line will run next.
4. **Step 4:** Click **Step** to run one line at a time; watch variable values in the **Locals** pane.

Button	**What it does**
Go	Run at normal speed until the next breakpoint (or input, or the end)
Step	Run the next line; enters a function call
Over	Run the next line; runs a function call without showing its details
Out	Finish the current function and return
Quit	Stop the program

Preconditions for Effective Debugging

1. **Understand the algorithm:** Know what the program is supposed to do, so you can write test cases.
2. **Check correctness:** Check the conditions and values before and after each step.
3. **Code tracing:** Follow the program by hand, line by line, writing down variable values.
4. **Peer review:** Ask a classmate to read your code; a fresh eye finds blind spots.
5. **Test with different inputs:** positive, negative, zero, and very large values.
6. **Tip:** If stuck, take a short break and come back — and ask why the program is doing **what it does**, not just why it is not working.

Lab 1: Student Details, Total, and Average

Read Student Name, USN, and marks of three courses; calculate total and average; display them neatly.

```
name = input("Enter student name: ")
usn = input("Enter USN: ")
m1 = float(input("Enter marks in course 1: "))
m2 = float(input("Enter marks in course 2: "))
m3 = float(input("Enter marks in course 3: "))

total = m1 + m2 + m3
average = total / 3

print("\n------- Student Details -------")
print("Name    :", name)
print("USN     :", usn)
print("Marks   :", m1, m2, m3)
print("Total   :", total)
print("Average :", format(average, ".2f"))
```

Sample Output

```
Enter student name: Ananya K
Enter USN: 4MT26CS001
Enter marks in course 1: 85
Enter marks in course 2: 78
Enter marks in course 3: 92

------- Student Details -------
Name    : Ananya K
USN     : 4MT26CS001
Marks   : 85.0 78.0 92.0
Total   : 255.0
Average : 85.00
```

Lab 2: Area and Circumference of a Circle

Read the radius; calculate area and circumference; convert the area into an integer.

```
radius = float(input("Enter the radius of the
    circle: "))
PI = 3.14159

area = PI * radius * radius
circumference = 2 * PI * radius

print("Area of the circle           =", format(
    area, ".2f"))
print("Circumference of the circle =", format(
    circumference, ".2f"))
print("Area as an integer           =", int(area))
```

Sample Output

```
Enter the radius of the circle: 7
Area of the circle           = 153.94
Circumference of the circle = 43.98
Area as an integer           = 153
```

1. **Concepts used:** `float()` conversion, arithmetic operators, `format()`, and `int()` truncation.
2. **Note:** `int(153.94)` gives `153` (decimal part removed), while `round(153.94)` would give `154`.

Lab 3: Values and Data Types — Program

Accept an integer, a float, a string, and a Boolean; display values and types; perform suitable operations.

```
i = int(input("Enter an integer: "))
f = float(input("Enter a floating-point number: "))
s = input("Enter a string: ")
b = input("Enter a Boolean (True/False): ") == "True"

print("\nValue :", i, " Type :", type(i))
print("Value :", f, " Type :", type(f))
print("Value :", s, " Type :", type(s))
print("Value :", b, " Type :", type(b))

print("\nInteger + Float   =", i + f)
print("Integer * 2       =", i * 2)
print("Float / 2         =", f / 2)
print("String + ' MITE'  =", s + " MITE")
print("String * 3        =", s * 3)
print("Length of string  =", len(s))
print("Boolean + 1       =", b + 1)
```

1. `int()` and `float()` convert the typed text into numbers.
2. The Boolean is obtained by **comparing** the text with `"True"` (see Slide 73).
3. `type()` shows the data type of each value.
4. `+` adds numbers but **joins** strings; `*` repeats a string.
5. `len()` gives the number of characters in a string.

Lab 3: Values and Data Types — Sample Output

Sample Output

```
Enter an integer: 25
Enter a floating-point number: 3.5
Enter a string: Python
Enter a Boolean (True/False): True

Value : 25  Type : <class 'int'>
Value : 3.5  Type : <class 'float'>
Value : Python  Type : <class 'str'>
Value : True  Type : <class 'bool'>

Integer + Float   = 28.5
Integer * 2       = 50
Float / 2         = 1.75
String + ' MITE'  = Python MITE
String * 3        = PythonPythonPython
Length of string  = 6
Boolean + 1       = 2
```

1. `25 + 3.5` gives `28.5`: the integer is converted to a float automatically (**implicit conversion**).
2. `3.5 / 2` gives `1.75`.
3. `"Python" * 3` repeats the string three times.
4. `True + 1` gives 2, because `True` behaves as `1` in arithmetic.

Lab 3: Why Not `bool(input())`?

```
>>> bool("True")
True
>>> bool("False")
True
>>> bool("")
False
```

```
>>> "False" == "True"
False
>>> "True" == "True"
True
```

1. `bool()` of **any non-empty string** is `True` — even the string `"False"`!
2. So in Lab 3 we **compare** the input with `"True"`: the result of `==` is a real Boolean.
3. This is a good example of a **logical error**: `bool(input())` runs without any error message but gives the wrong answer.

Lab 4: Arithmetic Operations on Two Numbers

Accept two numbers; perform addition, multiplication, division, floor division, modulus, and exponentiation.

```
a = int(input("Enter the first number: "))
b = int(input("Enter the second number: "))

print("Addition       :", a, "+", b, "=", a + b)
print("Multiplication :", a, "*", b, "=", a * b)
print("Division       :", a, "/", b, "=", a / b)
print("Floor Division :", a, "//", b, "=", a // b
    )
print("Modulus        :", a, "%", b, "=", a % b)
print("Exponentiation :", a, "**", b, "=", a ** b
    )
```

Sample Output

```
Enter the first number: 17
Enter the second number: 5
Addition       : 17 + 5 = 22
Multiplication : 17 * 5 = 85
Division       : 17 / 5 = 3.4
Floor Division : 17 // 5 = 3
Modulus        : 17 % 5 = 2
Exponentiation : 17 ** 5 = 1419857
```

1. **Try:** Enter `0` as the second number. Which lines fail, and with what error?

Summary

1. Python is a **high-level, interpreted, interactive, object-oriented**, open-source language created by **Guido van Rossum** (1991).
2. A program is written in a `.py` file and run with **F5**; it usually follows **Input** $\rightarrow$ **Processing** $\rightarrow$ **Output**.
3. **Literals** are fixed values; **variables** are named memory locations; **identifiers** follow naming rules; **keywords** are reserved.
4. `input()` always returns a **string**; `print()` displays values.
5. Basic data types: `int`, `float`, `complex`, `str`, `bool`.
6. Operators are evaluated by **precedence** and **associativity**.
7. **Type conversion** is implicit (coercion) or explicit (casting).
8. **Debugging:** read tracebacks, use `print()`, `assert`, and the IDLE debugger.

Review Questions

1. Explain any eight features of Python.
2. Write a short note on the history, applications, and limitations of Python.
3. Explain the steps to write and execute a Python program. Explain comments and indentation.
4. What are literal constants? Explain numeric and string literals with examples.
5. What is an identifier? State the rules for naming identifiers with valid and invalid examples.
6. Explain the `input()` and `print()` functions with examples.
7. Explain arithmetic, comparison, and logical operators with examples.
8. Explain operator precedence and associativity with examples.
9. What is type conversion? Differentiate implicit and explicit conversion.
10. What is debugging? Explain the types of errors and the debugging aids in Python.